

ACH2043

INTRODUÇÃO À TEORIA DA COMPUTAÇÃO

Processamento de GLCs:
Algoritmo CYK (Cocke–Younger–Kasami)

Referência:

SIPSER, M. Introdução à Teoria da Computação.
2ª edição, Ed. Thomson

Prof. Marcelo S. Lauretto (marcelolauretto@usp.br)

Introdução

- Na aula anterior:
 - Forma Normal de Chomsky
- Nesta aula:
 - Algoritmo para processamento de GLCs
 - CYK (Cocke–Younger–Kasami)
 - Apresentado na Seção 7.2 do livro-texto da disciplina
 - Objetivo: dada uma gramática G na Forma Normal de Chomsky e uma cadeia w , determinar se $w \in L(G)$

Forma Normal de Chomsky

Recapitulação da aula anterior:

- **Definição 2.8:**

Uma GLC está na Forma Normal de Chomsky (FNC) se:

a) Toda regra de substituição é da forma:

$$V \rightarrow W X \quad \text{ou}$$

$$V \rightarrow a$$

onde W, X são variáveis, a é um símbolo terminal

b) A variável inicial S_0 não pode aparecer no lado direito de nenhuma regra

c) Somente a variável inicial pode ter a regra

$$S_0 \rightarrow \varepsilon$$

Forma Normal de Chomsky

- Propriedades importantes da FNC:

- Dada uma gramática G na FNC com variável inicial S_0 :

1. A gramática G gera a cadeia vazia ε se e somente se possuir a regra $S_0 \rightarrow \varepsilon$
2. A gramática G gera uma cadeia unitária a se e somente se possuir a regra $S_0 \rightarrow a$

- Na verdade, esta é uma propriedade mais geral:

Uma variável V deriva a cadeia a (relembrando a notação: $V \xRightarrow{*} a$) se e somente se G possui a regra $V \rightarrow a$

- Dadas as propriedades 1 e 2, determinar se a gramática G gera a cadeia de entrada w torna-se trivial nos casos $w = \varepsilon$ ou $w = a$:

- Basta verificar a existência da regra $S_0 \rightarrow \varepsilon$ (se $w = \varepsilon$) ou da regra $S_0 \rightarrow a$ (se $w = a$)
 - Na gramática de exemplo ao lado, é possível verificar rapidamente que ela gera a cadeia unitária a , mas não a cadeia unitária b nem a cadeia vazia ε

$$\begin{aligned} S_0 &\rightarrow AA_1 \mid A_2B \mid a \mid AS \mid SA \\ S &\rightarrow AA_1 \mid A_2B \mid a \mid AS \mid SA \\ A &\rightarrow b \mid AA_1 \mid A_2B \mid a \mid AS \mid SA \\ A_1 &\rightarrow SA \\ A_2 &\rightarrow a \\ B &\rightarrow b \end{aligned}$$

Forma Normal de Chomsky

- Propriedades importantes da FNC:

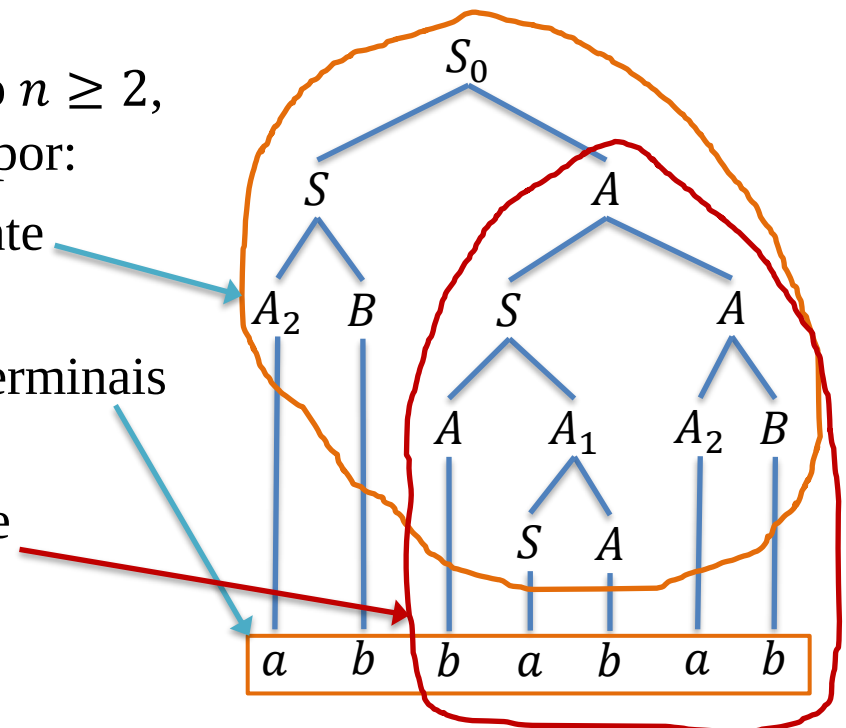
- Dada uma gramática G na FNC com variável inicial S_0 :

$$\begin{aligned} S_0 &\rightarrow AA_1 \mid A_2B \mid a \mid AS \mid SA \\ S &\rightarrow AA_1 \mid A_2B \mid a \mid AS \mid SA \\ A &\rightarrow b \mid AA_1 \mid A_2B \mid a \mid AS \mid SA \\ A_1 &\rightarrow SA \\ A_2 &\rightarrow a \\ B &\rightarrow b \end{aligned}$$

3. Para qualquer cadeia $w \in L(G)$ de comprimento $n \geq 2$, a árvore sintática de G sobre w será constituída por:

- Uma árvore binária com nós rotulados somente por variáveis
- Uma camada final com nós rotulados pelos terminais

- Propriedade recursiva: todas as subárvores sobre subcadeias de comprimento $n \geq 2$ têm essa mesma estrutura



Forma Normal de Chomsky

- Propriedades importantes da FNC:

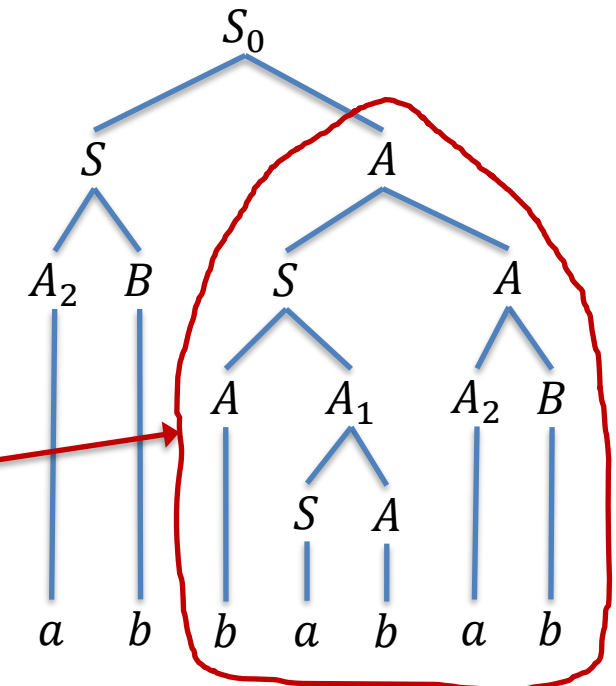
- Dada uma gramática G na FNC com variável inicial S_0 :

4. Para qualquer cadeia w de comprimento $n \geq 2$, uma variável V de G deriva a subcadeia $w_i w_{i+1} \dots w_j$ ($j > i$) se e somente se existem duas variáveis V_1 e V_2 tais que:

- $V_1 \xRightarrow{*} w_i \dots w_k$ e $V_2 \xRightarrow{*} w_{k+1} \dots w_j$, para algum $k \in \{i, i+1, \dots, j-1\}$
- G possui uma regra $V \rightarrow V_1 V_2$

- Exemplo: na árvore sintática ao lado, $A \xRightarrow{*} babab$ porque:

- $S \xRightarrow{*} bab$ e $A \xRightarrow{*} ab$
- $A \Rightarrow SA$, graças à regra $A \rightarrow SA$

$$\begin{aligned} S_0 &\rightarrow AA_1 \mid A_2 B \mid a \mid AS \mid SA \\ S &\rightarrow AA_1 \mid A_2 B \mid a \mid AS \mid SA \\ A &\rightarrow b \mid AA_1 \mid A_2 B \mid a \mid AS \mid SA \\ A_1 &\rightarrow SA \\ A_2 &\rightarrow a \\ B &\rightarrow b \end{aligned}$$


Algoritmo CYK

- Cocke–Younger–Kasami
 - Método eficiente para parsing de GLCs, baseado na propriedade 4
 - Programação dinâmica: Decomposição de um problema complexo em subproblemas mais simples
- Estrutura de dados principal: Matriz triangular superior M
 - $M(i, j) =$ lista (ou conjunto) das variáveis de G que derivam a subcadeia $w_i w_{i+1} \dots w_j$ ($j \geq i$)
 - Ex: na figura abaixo:
 $M(4,7) =$ lista de variáveis de G que derivam a subcadeia $w_4 w_5 w_6 w_7$

	1	2	3	4	5	6	7	8
	w_1	w_2	w_3	w_4	w_5	w_6	w_7	w_8
1								
2								
3								
4							$M(4,7)$	
5								
6								
7								
8								

Posições não utilizadas

Algoritmo CYK

- Preenchimento de M :

- Preenchimento de M é feito em ordem crescente dos tamanhos das subcadeias de w

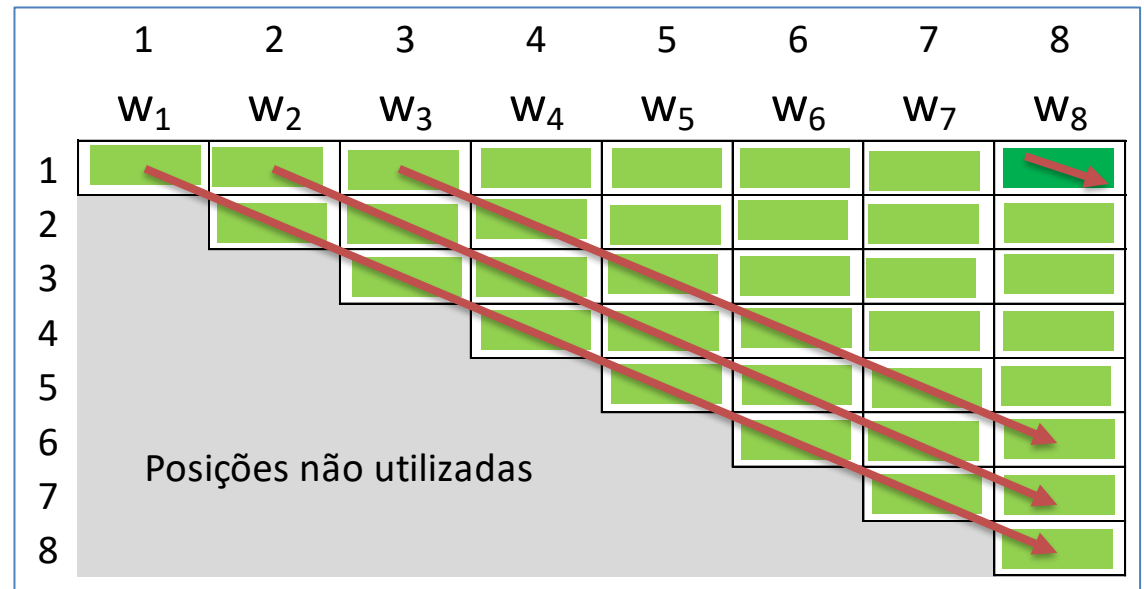
- Subcadeias de tamanho 1

- Subcadeias de tamanho 2

- Subcadeias de tamanho 3

- ...

- Cadeia completa $w_1 w_2 \dots w_n$
 $M(1, n)$



- Após o preenchimento de M , podemos determinar a aceitação ou não da cadeia:

- Se $S_0 \in M(1, n)$, *aceite*; senão, *rejeite*

Algoritmo CYK: Preenchimento de M

- Utilizaremos o exemplo abaixo para ilustrar o método:

GLC para a linguagem das cadeias com mesma quantidade de a's e b's

$$S \rightarrow aSb \mid bSa \mid SS \mid \varepsilon$$

Conversão para FNC:

1) Criação de nova variável inicial:

$$S_0 \rightarrow S$$

$$S \rightarrow aSb \mid bSa \mid SS \mid \varepsilon$$

2) Eliminação de substituições ε :

$$S_0 \rightarrow S \mid \varepsilon$$

$$S \rightarrow aSb \mid bSa \mid SS \mid ab \mid ba \mid S$$

3) Eliminação de regras unitárias:

$$S_0 \rightarrow aSb \mid bSa \mid SS \mid ab \mid ba \mid \varepsilon$$

$$S \rightarrow aSb \mid bSa \mid SS \mid ab \mid ba$$

4) Padronização:

$$S_0 \rightarrow AT \mid BU \mid SS \mid AB \mid BA \mid \varepsilon$$

$$S \rightarrow AT \mid BU \mid SS \mid AB \mid BA$$

$$T \rightarrow SB$$

$$U \rightarrow SA$$

$$A \rightarrow a$$

$$B \rightarrow b$$

Algoritmo CYK: Preenchimento de M

- Subcadeias de tamanho 1:

- Para $i = 1$ até n :
 - Denote $b \equiv w_i$
 - Para toda regra $V \rightarrow b$ (na qual b apareça no lado direito), adicione a variável V em $M(i, i)$

$S_0 \rightarrow AT \mid BU \mid SS \mid AB \mid BA \mid \varepsilon$
 $S \rightarrow AT \mid BU \mid SS \mid AB \mid BA$
 $T \rightarrow SB$
 $U \rightarrow SA$
 $A \rightarrow a$
 $B \rightarrow b$

	1	2	3	4	5	6
	a	a	b	a	b	b
$i = 1$	1					
$i = 2$	2					
$i = 3$	3					
$i = 4$	4					
$i = 5$	5					
$i = 6$	6					

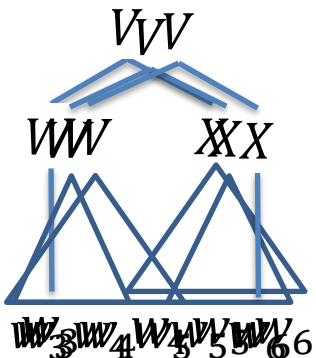
Algoritmo CYK: Preenchimento de M

- Subcadeias de tamanho $l \geq 2$:
 - Para $l = 2$ até n : [comprimento da subcadeia]
 - Para $i = 1$ até $n - l + 1$: [posição inicial]
 - $j = i + l - 1$ [posição final]
 - Para $k = i$ até $j - 1$: [ponto da divisão]
 - Para toda regra da forma $V \rightarrow WX$ onde $W \in M(i, k)$ e $X \in M(k + 1, j)$, adicione a variável V em $M(i, j)$

Exemplo: preenchimento de $M(3,6)$

	1 w_1	2 w_2	3 w_3	4 w_4	5 w_5	6 w_6
1						
2						
3			W	W	W	$M(3,6)$
4			k			X
5						X
6						X

$$l = 4, i = 3, k = 5$$



$\forall W \in M(3,5), \forall X \in M(6,6):$
 Se $\exists$ regra $V \rightarrow WX \Rightarrow$
 $M(3,6) = M(3,6) \cup \{V\}$

Algoritmo CYK: Preenchimento de M

- Subcadeias de tamanho $l \geq 2$:
 - Para $l = 2$ até n : [comprimento da subcadeia]
 - Para $i = 1$ até $n - l + 1$: [posição inicial]
 - $j = i + l - 1$ [posição final]
 - Para $k = i$ até $j - 1$: [ponto da divisão]
 - Para toda regra da forma $V \rightarrow WX$ onde $W \in M(i, k)$ e $X \in M(k + 1, j)$, adicione a variável V em $M(i, j)$

Exemplo: GLC

$S_0 \rightarrow AT \mid BU \mid SS \mid \boxed{AB} \mid BA \mid \varepsilon$
 $S \rightarrow AT \mid BU \mid SS \mid \boxed{AB} \mid BA$
 $T \rightarrow SB$
 $U \rightarrow SA$
 $A \rightarrow a$
 $B \rightarrow b$

$? \rightarrow AA?$

$l = 2, i = 1, j = 2, k = 1$
 $l = 2, i = 2, j = 3, k = 2$
 $l = 2, i = 3, j = 4, k = 3$
 $l = 2, i = 4, j = 5, k = 4$
 $l = 2, i = 5, j = 6, k = 5$

	1	2	3	4	5	6
	a	a	b	a	b	b
1	A	$\emptyset$				
2		A	S_0, S			
3			B	S_0, S		
4				A	S_0, S	
5					B	$\emptyset$
6						B

Algoritmo CYK: Preenchimento de M

- Subcadeias de tamanho $l \geq 2$:
 - Para $l = 2$ até n : [comprimento da subcadeia]
 - Para $i = 1$ até $n - l + 1$: [posição inicial]
 - $j = i + l - 1$ [posição final]
 - Para $k = i$ até $j - 1$: [ponto da divisão]
 - Para toda regra da forma $V \rightarrow WX$ onde $W \in M(i, k)$ e $X \in M(k + 1, j)$, adicione a variável V em $M(i, j)$

Exemplo: GLC

$S_0 \rightarrow AT \mid BU \mid SS \mid AB \mid BA \mid \varepsilon$
 $S \rightarrow AT \mid BU \mid SS \mid AB \mid BA$
 $T \rightarrow SB$
 $U \rightarrow SA$
 $A \rightarrow a$
 $B \rightarrow b$

$? \rightarrow AS?$

$? \rightarrow AS_0?$

$l = 3, i = 1, j = 3$ $k = 2$

$l = 3, i = 2, j = 4$ $k = 3$

$l = 3, i = 3, j = 5$

$l = 3, i = 4, j = 6$

$S_0 \in M(1, n) \Rightarrow$ cadeia aceita

	1	2	3	4	5	6
	a	a	b	a	b	b
1	A	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	S_0, S
2		A	S_0, S	U	S_0, S	T
3			B	S_0, S	T	$\emptyset$
4				A	S_0, S	T
5					B	$\emptyset$
6						B

Algoritmo CYK consolidado

- Entrada: Gramática G na FNC com variável inicial S_0 e cadeia $w = w_1 \dots w_n$
 1. Se $w = \varepsilon$, verifique se G possui a regra $S_0 \rightarrow \varepsilon$; se sim, *aceite*; senão, *rejeite*.
 2. Se $w = a$, onde a é um terminal, verifique se G possui a regra $S_0 \rightarrow a$; se sim, *aceite*; senão, *rejeite*.
 3. Para $i = 1$ até n : [tratamento de subcadeias de tamanho 1]
 4. Denote $b \equiv w_i$
 5. Para toda regra $V \rightarrow b$ (na qual b apareça no lado direito), adicione V em $M(i, i)$
 6. Para $l = 2$ até n : [tratamento de subcadeias de tamanho $l \geq 2$]
 7. Para $i = 1$ até $n - l + 1$: [i : posição inicial da subcadeia]
 8. Faça $j = i + l - 1$ [j : posição final da subcadeia]
 9. Para $k = i$ até $j - 1$: [k : posição para quebra da subcadeia]
 10. Para toda regra da forma $V \rightarrow WX$, onde $W \in M(i, k)$ e $X \in M(k + 1, j)$, adicione a variável V em $M(i, j)$
 11. Se S_0 estiver em $M(1, n)$, *aceite*; senão, *rejeite*.

Comentários finais

1. Complexidade de tempo do algoritmo:

Notação: n = comprimento da cadeia; r = número de regras de G ; v = número de variáveis

- Linhas 3-5: laço com n repetições, cada uma com custo $O(r)$
 - Complexidade resultante: $O(rn)$
- Linhas 6-10:
 - Três laços aninhados, com $O(n)$ em cada nível $\Rightarrow O(n^3)$ repetições;
 - Cada execução da linha 10: custo $O(r)$
 - Complexidade resultante: $O(rn^3)$
- Linha 11: custo $O(1)$
- Complexidade final:
 $O(rn) + O(rn^3) + O(1) = O(rn^3)$ (ou $O(n^3)$, se $r \ll n$)

Comentários finais

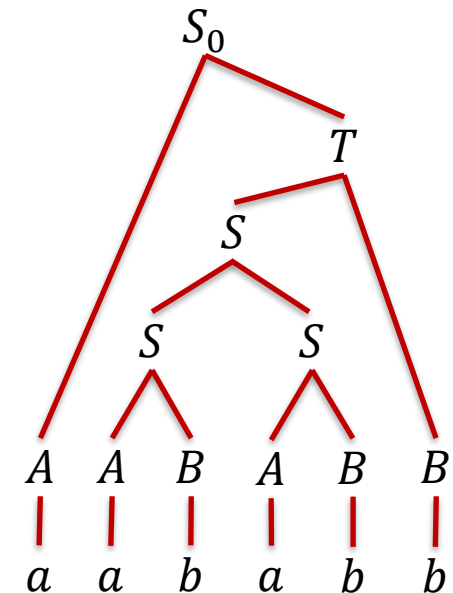
2. Este método fornece também todas as subcadeias x de w que pertencem a $L(G)$
 - Útil para buscar todas as ocorrências de um certo padrão em cadeias de tamanho moderado (Mas complexidade de tempo pode ser um limitante)
 - Basta verificar em quais posições de M aparece a variável inicial
 - No exemplo ao lado, M permite identificar rapidamente todas as subcadeias com as mesmas quantidades de a's e b's

	1	2	3	4	5	6
	a	a	b	a	b	b
1	A	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	S_0, S
2		A	S_0, S	U	S_0, S	T
3			B	S_0, S	T	$\emptyset$
4				A	S_0, S	T
5					B	$\emptyset$
6						B

Comentários finais

3. A versão aqui apresentada do algoritmo é um *reconhecedor*, mas *não fornece* a(s) árvore(s) sintática(s)
- Para esta finalidade:
 - M precisaria incorporar, para cada nova variável V inserida, os ponteiros para as posições de M referentes à derivação e suas respectivas variáveis W e X
 - Uma vez preenchida a tabela, faz-se uma busca em profundidade/largura sobre uma das possíveis árvores possíveis a partir da variável inicial em $M(1, n)$
 - Ilustração:

	1	2	3	4	5	6
	a	a	b	a	b	b
1	$A \leftarrow$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	S_0, S
2		$A \leftarrow$	$S_0, S \leftarrow$	U	$S_0, S \leftarrow$	T
3			$B \leftarrow$	S_0, S	T	$\emptyset$
4				$A \leftarrow$	S_0, S	T
5					$B \leftarrow$	$\emptyset$
6						B



- Obs: Reconstrução das árvores *não é* necessária em nosso exercício de programação